

Tools of the Trade: Linux and SQL

Module 1
Introduction to operating systems

Module 2
The Linux operating system

Module 3
Linux commands in the Bash shell

Module 4
Databases and SQL

Introduction to SQL and Databases

✔ Welcome to module 4
Video • 1 min

✔ Introduction to databases
Video • 3 min

✔ Query databases with SQL
Video • 2 min

✔ SQL filtering versus Linux filtering
Reading • 4 min

✔ A day's SQL in cybersecurity
Video • 2 min

✔ Test your knowledge: SQL and databases
Practice Assignment • Grade: 100%

SQL queries

✔ Basic queries
Video • 3 min

✔ Query a database
Reading • 8 min

✔ Resources for completing SQL labs
Reading • 4 min

✔ Activity: Perform a SQL query
Ungraded App Item • 30 min

Optional Exemplar: Perform a SQL query
Ungraded App Item • 10 min

✔ Basic filters on SQL queries
Video • 5 min

✔ Exemplar: Perform an SQL query
Reading • 10 min

✔ The WHERE clause and basic operators
Reading • 8 min

✔ Activity: Filter a SQL query
Ungraded App Item • 30 min

Optional Exemplar: Filter a SQL query
Ungraded App Item • 10 min

✔ Exemplar: Filter a SQL query
Reading • 10 min

✔ Test your knowledge: SQL queries
Practice Assignment • Grade: 100%

More SQL filters

✔ Filter dates and numbers
Video • 4 min

✔ Operators for filtering dates and numbers
Reading • 4 min

✔ Activity: Apply more filters in SQL
Ungraded App Item • 30 min

Optional Exemplar: Apply more filters in SQL
Ungraded App Item • 10 min

Exemplar: Apply more filters in SQL
Reading • 10 min

Filters with AND, OR, and NOT
Video • 5 min

More on filters with AND, OR, and NOT
Reading • 8 min

Activity: Filter with AND, OR, and NOT
Ungraded App Item • 30 min

Optional Exemplar: Filter with AND, OR, and NOT
Ungraded App Item • 10 min

Exemplar: Filter with AND, OR, and NOT
Reading • 10 min

Portfolio Activity: Apply filters to SQL queries
Practice Assignment • 30 min

Portfolio Activity Exemplar: Apply filters to SQL queries
Reading • 4 min

Test your knowledge: More SQL filters
Practice Assignment • 8 min

SQL joins

Join tables in SQL
Video • 5 min

Types of joins
Video • 2 min

Compare types of joins
Reading • 8 min

Identify: Choose the appropriate type of join
Ungraded Plugin • 10 min

Activity: Complete a join
Ungraded App Item • 30 min

Optional Exemplar: Complete a join
Ungraded App Item • 10 min

Exemplar: Complete a join
Reading • 10 min

Test your knowledge: SQL joins
Practice Assignment • 8 min

Continuous learning in SQL
Reading • 8 min

Review: Databases and SQL

Coach dialogue: Explore SQL applications in security
Dialogue • 15 min

Wrap-up
Video • 1 min

Reference guide: SQL
Reading • 2 min

Glossary terms from module 4
Reading • 2 min

Module 4 challenge
Graded Assignment • 50 min

Congratulations on completing Course 4!

Reflect and connect with peers
Reading • 2 min

Course wrap-up
Video • 1 min

Course 4 glossary

Exemplar: Apply more filters in SQL

00 0:39 / 4:07

Activity Overview

This exemplar provides a detailed walk through and solutions for the "Apply more filters in SQL" lab activity. As a security analyst, you'll often need to refine your data retrieval by filtering based on specific dates, times, and ranges. This exemplar will guide you through using SQL operators like `>`, `>=`, `<`, `<=`, `BETWEEN`, and filtering by specific IDs.

Scenario

Continuing your work with the organization database, your team requires more precise information from the login attempt logs. You need to retrieve records based on specific time frames and individual login identifiers for investigation and analysis.

Task 1. Filter for login attempts made after a certain date

Your team is interested in login attempts that occurred after January 15th, 2023, to investigate recent activity. The `login_date` column in the `log_in_attempts` table stores the date of each attempt.

Use the `>` (greater than) operator to filter for records where the `login_date` is after the specified date. Replace `YYYY-MM-DD` with the correct date.

```
1 SELECT *
2 FROM log_in_attempts
3 WHERE login_date > 'YYYY-MM-DD';
```

The correct query to solve this step:

```
1 SELECT *
2 FROM log_in_attempts
3 WHERE login_date > '2023-01-15';
```

How many login attempts occurred after January 15th, 2023?

Answer: There were [Insert Example Answer Here, e.g., 112] login attempts after January 15th, 2023.

Task 2. Filter for login attempts made in a certain date range

Your team needs to analyze login activity during the first week of February 2023, specifically from February 1st to February 7th (inclusive). The `login_date` column in the `log_in_attempts` table contains the date.

Use the `BETWEEN` operator to filter for records within the specified start and end dates. Replace `YYYY-MM-DD` with the correct dates.

```
1 SELECT *
2 FROM log_in_attempts
3 WHERE login_date BETWEEN 'YYYY-MM-DD' AND 'YYYY-MM-DD';
```

The correct query to solve this step:

```
1 SELECT *
2 FROM log_in_attempts
3 WHERE login_date BETWEEN '2023-02-01' AND '2023-02-07';
```

How many login attempts were made between February 1st and February 7th, 2023?

Answer: There were [Insert Example Answer Here, e.g., 88] login attempts in that date range.

Task 3. Filter for login attempts made at a certain time

Your team is investigating a potential issue that occurred around 09:30 AM. You need to retrieve all login attempts that happened at exactly this time. The `login_time` column in the `log_in_attempts` table stores the time of each attempt.

Use the `=` (equals) operator to filter for records where the `login_time` matches the specific time. Replace `HH:MM:SS` with the correct time.

```
1 SELECT *
2 FROM log_in_attempts
3 WHERE login_time = 'HH:MM:SS';
```

The correct query to solve this step:

```
1 SELECT *
2 FROM log_in_attempts
3 WHERE login_time = '09:30:00';
```

How many login attempts occurred at 09:30:00?

Answer: There were [Insert Example Answer Here, e.g., 5] login attempts at 09:30:00.

Task 4. Filter for login attempts by ID

You need to retrieve the details of a specific login attempt with the `login_id` of 503. The `login_id` column in the `log_in_attempts` table uniquely identifies each login attempt.

Use the `=` (equals) operator to filter for the record where the `login_id` matches the specified value. Replace `ID_Value` with the correct ID.

```
1 SELECT *
2 FROM log_in_attempts
3 WHERE login_id = ID_Value;
```

The correct query to solve this step:

```
1 SELECT *
2 FROM log_in_attempts
3 WHERE login_id = 503;
```

What was the `login_date` for the login attempt with `login_id` 503?

Answer: The `login_date` for login ID 503 was [Insert Example Answer Here, e.g., 2023-02-10].

Conclusion

Excellent work! You've now practiced applying more specific filters in SQL using comparison operators `>`, `>=`, `<`, `<=`, `=`, and the `BETWEEN` operator to retrieve data based on dates, times, and unique identifiers. These skills are essential for conducting thorough investigations and extracting targeted information from security logs and databases. You are becoming more adept at using SQL to analyze and understand security-related data.

Like Dislike Report an issue

Mark as completed

coach

Hi, Venkatesh!
How can I help?

Explain this topic in simple terms Give me a summary

Give me practice questions Give me real-life examples

Ask me anything

Coach is powered by AI, so check for mistakes and don't share sensitive info. Your data will be used in accordance with Coursera's Privacy Notice.

<https://www.coursera.org/learn/linux-and-sql/supplement/UAQR1/exemplar-apply-more-filters-in-sql>

1/1